

420-SN1-RE Programming in Science - Exercises for Topic 4 - While/If

These are practice questions *similar* to what you *might* see on a quiz or exam. Remember you will not have access to a computer during a test, so try to answer these without referring to Python. Once you've tried to answer the questions, feel free to check your answers with the Python environment.

A. Answer the following multiple choice questions:

1. How many times would the following program print Hello?

```
i = 1
while i < 10:
    if i < 3:
        print("Hello")
    elif i >= 8:
        print("Hello")
    i += 1
```

1
2
3

a. 10

c. 4

b. 5

d. 3

2. How many times would the following program print Hello?

```
x = 5
while x >= 0:
    if x % 2 == 0:
        print("Hello")
    x -= 1
```

1
2
3

a. 6

c. 3

b. 5

d. 0

3. How many times would the following program print Hello?

```
i = 0
while i < 3:
    j = 0
    while j < 3:
        if i >= j:
            print("Hello")
        j += 1
    i += 1
```

1

a. 12

c. 6

b. 9

d. 3

B. Write a program that does each of the following:

1. The “hailstone sequence”, also known as the Collatz sequence, is defined for any integer n as follows:

- If $n = 1$, the sequence ends.
- If n is even, the next number in the sequence is $n/2$.
- If n is odd, the next number in the sequence is $3n + 1$.

For every n that has been checked empirically, this sequence always reaches 1. However, there is no proof that this is always true.

Write a program to prompt the user to enter an integer for n , then print the numbers in the hailstone sequence, beginning with n and ending when n reaches 1.

Be sure your program never creates or prints **float** values!

As an extra exercise, count the number of integers your program prints, and print the total number of integers in the sequence. For example, starting at $n = 7$, the sequence consists of the following 17 numbers:

7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2, 1

2. Write a program that repeatedly asks a user to enter a **float** number. If this number is zero, the program **should stop** and print both the average and the *sum of the squares* of the previously-entered numbers. Round the results to the **nearest two digits**. For example, here is an example run of the program, where what the program prints is in black, and what the user types is in **red**.

Enter a real number: **1.5**
Enter a real number: **3**
Enter a real number: **2.5**
Enter a real number: **0**

Average: 2.33

Sum squared: 17.5

This is because the average is $(1.5 + 3 + 2.5)/3 = 7/3$, and $1.5^2 + 3^2 + 2.5^2 = 17.5$. Note that the final zero should not affect the average calculation. You do *not* need to use lists to do this computation!

3. *This problem is adapted from the Canadian Computing Competition, 2005 (Problem J2).*

When a credit card number is sent through the Internet it must be protected so that other people cannot see it. Many web browsers use a protection based on “RSA Numbers.” A number is an RSA number if it has exactly four divisors, including 1 and the number itself. In other words, a number is considered an RSA number if there are exactly four numbers that divide into it evenly. For example, 10 is an RSA number because it has exactly four divisors (1, 2, 5, 10). 12 is not an RSA number because it has too many divisors (1, 2, 3, 4, 6, 12). 11 is not an RSA number either. There is only one RSA number in the range 10...12. Write a program that inputs a range of numbers and then counts how many numbers from that range are RSA numbers. You may assume that the numbers in the range are less than 1000.

Sample Session 1

Enter lower limit of range: 10
Enter upper limit of range: 12
The number of RSA numbers between 10 and 12 is 1

Sample Session 2

Enter lower limit of range: 11

Enter upper limit of range: 15

The number of RSA numbers between 11 and 15 is 2